CHAPTER 1

INTRODUCTION

1.1 Overview

Mobile robots are increasingly used in indoor environments for inspection, assistance, delivery, monitoring, and research. For a robot to be useful to a non-technical user, the interaction method should be simple and natural. Instead of requiring coordinates, joystick control, or programming commands, a user should be able to provide an instruction such as "Find the green marker" or, in later development, "Go to the door".

Natural-language mobile robot navigation is challenging because a human instruction must be converted into information that a robot can understand and execute. The robot must identify the intended target, connect the instruction with its current camera view, choose an appropriate movement action, and stop safely when the action is uncertain or an obstacle is detected.

This Final Year Project investigates how structured prompt engineering can support this process on one low-cost indoor mobile robot prototype. The finalized physical system uses a Raspberry Pi 4 for onboard prompt processing, USB-webcam perception, lightweight OpenCV visual grounding, action selection, and result logging. An ESP32 performs sensor reading, local safety checks, and four-motor control through two MX1508 motor-driver modules.

The first prototype intentionally targets the basic and repeatable goal `find the green marker`. It adapts the modular language-to-vision-to-action concept demonstrated by LM-Nav and the restricted prompt-based action-selection idea used by VLMnav. It does not attempt to reproduce a large autonomous navigation platform or run an unrestricted large vision-language model directly onboard.

1.2 Background and Motivation

Traditional mobile robot navigation commonly uses maps, localization, path planning, and sensor feedback. These methods are effective when the destination is represented by coordinates or predefined waypoints. However, ordinary users normally describe destinations using natural language and visual concepts.

Prompt engineering can constrain language interpretation into a structured response containing a target, landmarks, spatial relation, action goal, and uncertainty. Visual grounding can then connect the structured target to an image captured by the robot. For a physical robot, the output must also be restricted to approved actions and checked against local safety information.

The motivation for this project is therefore to investigate a practical division of responsibility. The Raspberry Pi 4 performs high-level language and visual processing, while the ESP32 performs time-sensitive sensor reading, motor commands, obstacle detection, and command-timeout safety. This architecture supports prompt-based robot navigation without allowing uncertain high-level output to bypass deterministic local safety rules.

1.3 Problem Statement

Natural-language instructions are intuitive for humans but are not directly usable by a mobile robot. Instructions may be ambiguous, incomplete, or dependent on objects visible in the environment. Free-form model responses are also difficult to connect reliably to motor control because they may contain unsupported actions or inconsistent wording.

Low-cost mobile robot platforms introduce further constraints. A USB webcam does not provide depth or a complete map. The Raspberry Pi 4 has limited resources for large vision-language models, so the first prototype requires a lightweight and measurable visual-grounding method. Two front ultrasonic sensors provide limited obstacle coverage, and the low-cost motor and power subsystems require careful integration.

There is therefore a need for a focused low-cost system that can convert a simple natural-language indoor goal into structured machine-readable information, ground a supported target using a webcam image, restrict the result to an approved action set, and validate the action using ESP32 safety information before movement.

1.4 Research Questions

1. How can structured prompt engineering convert simple indoor navigation instructions into consistent machine-readable targets, actions, and uncertainty?
2. How can Raspberry Pi 4 webcam-based visual grounding and ESP32 sensor feedback be combined to select and execute safe basic robot actions?

3. How reliably can the completed Raspberry Pi 4 prototype find and approach a supported visual landmark in a controlled indoor environment?

1.5 Aim and Objectives

The aim of this project is to design and implement a Raspberry Pi 4-based prompt-engineered mobile robot for basic navigation in a controlled indoor environment.

The objectives are:

1. To develop a structured prompt-engineering pipeline that converts a simple natural-language navigation instruction into fixed fields and an approved action set.
2. To integrate a Raspberry Pi 4, USB webcam, ESP32, two HC-SR04 sensors, GY-291 / ADXL345 accelerometer, two MX1508 motor drivers, four DC gear motors, and the completed protected power system into one physical robot.
3. To evaluate prompt validity, visual grounding, action correctness, latency, local safety behaviour, and simple physical navigation success.

1.6 Research Scope

The project is limited to one low-cost four-wheel indoor mobile robot using a Raspberry Pi 4 as the only onboard high-level compute platform. The robot operates in supervised controlled areas such as rooms, laboratories, and simple landmark test zones.

The first physical prototype focuses on the single-target goal `find the green marker`. The approved action set is limited to `move_forward`, `turn_left`, `turn_right`, `search`, and `stop`. The Raspberry Pi performs structured instruction processing, webcam capture, visual grounding, action selection, validation, and logging. The ESP32 reads the ultrasonic sensors and GY-291, controls four motors through two MX1508 modules, and enforces obstacle, sensor-fault, invalid-command, and command-timeout stop behaviour.

The GY-291 is used for X/Y/Z acceleration, gravity-based roll and pitch tilt, motion, vibration, and shock observations. It is not treated as a heading sensor. Advanced functions such as full simultaneous localization and mapping, precise coordinate navigation, wheel-encoder odometry, autonomous route planning, LiDAR, RGB-D perception, and unrestricted large-model control are outside the current scope.

A separate Docker-based laptop webcam demonstration may be used during FYP1 to illustrate expected action outputs before final physical testing. It is not a second physical project approach.

1.7 Significance of the Project

The project contributes a practical framework for connecting prompt engineering with a real low-cost robot. Structured output makes navigation decisions easier to parse, log, evaluate, and reject when unsafe. Separating Raspberry Pi high-level processing from ESP32 local control also demonstrates how model-based navigation can be combined with deterministic safety behaviour.

The first-prototype focus makes the research achievable and measurable. A reliable marker-navigation demonstration provides a baseline that can later be extended with object detection for chairs, doors, and signboards, relation-aware instructions, additional sensors, mapping, and stronger navigation models.

1.8 Organization of the Thesis

Chapter 1 introduces the project background, problem statement, research questions, objectives, scope, and significance. Chapter 2 reviews related work on language-guided navigation, prompt engineering, visual grounding, and suitable low-cost hardware. Chapter 3 presents the Raspberry Pi 4 methodology, system architecture, mechanical and electrical design, implementation procedure, and validation plan. Chapter 4 presents the expected FYP1 results and defines the physical measurements required during FYP2.

1.9 Summary

This chapter established the need for a focused and safety-aware approach to natural-language mobile robot navigation. The finalized project develops one Raspberry Pi 4 physical prototype with one ESP32 sensor, safety, and motor-control layer.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews literature related to Prompt Engineering for Mobile Robot Navigation. The project investigates how a mobile robot can receive a natural language instruction, extract useful navigation information, ground the instruction using visual input, and convert the result into simple movement commands. The work is positioned as a low-cost indoor Final Year Project (FYP) research prototype rather than an industry-level autonomous navigation system.

The main baseline paper for this project is LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action by Shah et al. (2023a). LM-Nav is important because it presents a modular architecture for language-guided navigation. Its pipeline separates the problem into language understanding, visual grounding, and navigation execution. This structure is suitable for this FYP because the same idea can be adapted into one focused low-cost Raspberry Pi 4 robot with ESP32 low-level control.

The project does not attempt to reproduce the complete hardware capability of LM-Nav or newer large-scale navigation systems. Instead, the project adapts the core research idea into a student-level Raspberry Pi 4 physical robot. The first prototype uses the basic instruction "Find the green marker". The Raspberry Pi 4 produces structured navigation information, uses the USB-webcam image and lightweight OpenCV grounding, validates the proposed action using ESP32 sensor status, and sends a simple command such as move_forward, turn_left, turn_right, stop, or search to the ESP32 motor-control layer.

Several related works are reviewed to clarify the project scope. VLMnav is included as the main prompt and action-selection reference because it formulates navigation as a question-answering or action-choice task. VLMaps and HOV-SG are reviewed because they improve spatial grounding, but they require map construction, RGB-D sensing, or scene graph construction, so they are treated as future work rather than the main implementation. ViNT and NoMaD are reviewed as stronger navigation execution models, but they are not the main focus because this FYP emphasizes prompt engineering, visual grounding, and simple action

control. More recent VLM and VLA navigation systems such as `NaVid`, `Uni-NaVid`, and `NaVILA` are also discussed as advanced research directions.

The aim of this chapter is therefore not only to describe related papers, but also to critically compare their suitability for a low-cost indoor robot prototype.

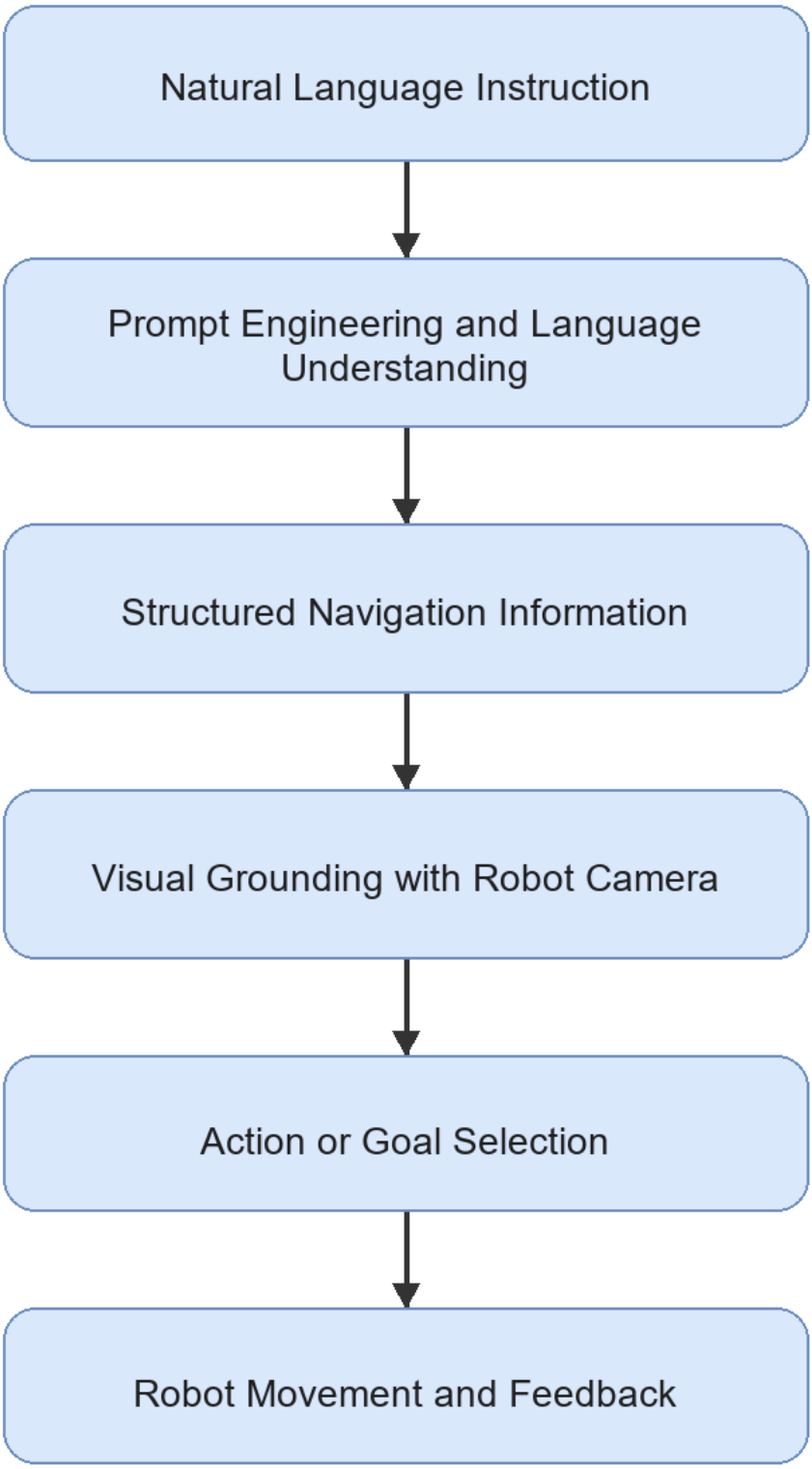
2.2 Mobile Robot Navigation and Natural Language Instructions

Mobile robot navigation normally requires a robot to perceive its environment, decide where to move, and execute motion safely. Traditional navigation systems often rely on a map, localization, path planning, obstacle detection, and low-level motor control. These systems are effective when the goal is specified in a robot-readable form, such as a coordinate, waypoint, or selected map location. However, this interaction style is not natural for non-technical users.

Natural language provides a more intuitive interface. A user can describe a goal using objects, places, and relations, for example "go to the door", "find the signboard", or "move toward the chair near the table". These instructions are simple for humans, but they are difficult for robots because they contain semantic meaning that must be connected to the physical environment.

Language-guided navigation therefore requires three main capabilities. First, the robot must understand the user's instruction. Second, it must ground the instruction in visual input or spatial knowledge. Third, it must convert the result into movement. This general process is shown in Figure 2.1.

Figure 2.1: General language-guided navigation process



For this FYP, the navigation problem is intentionally limited to indoor environments such as a corridor, room, laboratory, door area, table area, chair area, or signboard location. This

limitation is necessary because the proposed hardware uses a USB webcam and simple wheeled motion, not LiDAR, RGB-D SLAM, or high-cost navigation hardware. The controlled environment makes it possible to evaluate the core research question: whether structured prompt engineering can produce useful navigation information for a simple mobile robot.

2.3 Prompt Engineering for Robot Navigation

Prompt engineering is the design of model instructions, examples, constraints, and output formats so that an LLM or VLM produces useful responses. In robot navigation, prompt engineering is important because the model output must be connected to a physical robot. A free-form explanation is not enough. The output should contain machine-readable information such as the target object, relevant landmarks, spatial relation, action goal, suggested action, and uncertainty level.

In an LM-Nav-style system, prompt engineering can be used to extract landmarks from a natural language command. In a VLMnav-style system, prompt engineering can ask a model to choose the best next action from a fixed set. For a low-cost robot prototype, both ideas are useful. The robot does not need a complex high-level planner in the first implementation if the prompt output can be converted into a small set of validated motor commands.

Table 2.1: Roles of prompt engineering in mobile robot navigation

For this project, prompt engineering is treated as the interface between human language and robot action. The prompt must not only produce a correct sentence; it must produce an output that can be checked, logged, evaluated, and used by the onboard Raspberry Pi 4 decision logic.

2.4 Vision-Language Grounding for Indoor Landmarks

Vision-language grounding is the process of connecting text descriptions to visual observations. For mobile robot navigation, this means matching a target word such as "door", "chair", "table", "corridor", or "signboard" with what the robot sees through its camera.

CLIP and OpenCLIP-style models are common choices for image-text grounding because they embed images and text into a shared feature space. A text query can be compared with an image using cosine similarity:

```
score(image, text) = cosine_similarity(E_image(image), E_text(text))
```

where `E_image` is the image encoder and `E_text` is the text encoder. The image or region with the highest score can be treated as the most relevant match. A modern VLM may also be used by asking a question such as "Does this image contain a door?" or "Which action should the robot take to find the chair near the table?"

However, visual grounding remains a limitation for low-cost robots. A USB webcam provides only RGB images and does not directly provide depth, object distance, or a 3D map. The system may also struggle when multiple similar objects are visible, when lighting is poor, or when the target is partly outside the camera view. For this reason, the proposed project focuses on simple indoor landmarks and a small action set rather than full autonomous navigation in complex environments.

2.5 LM-Nav as the Main Baseline

2.5.1 Overview of LM-Nav

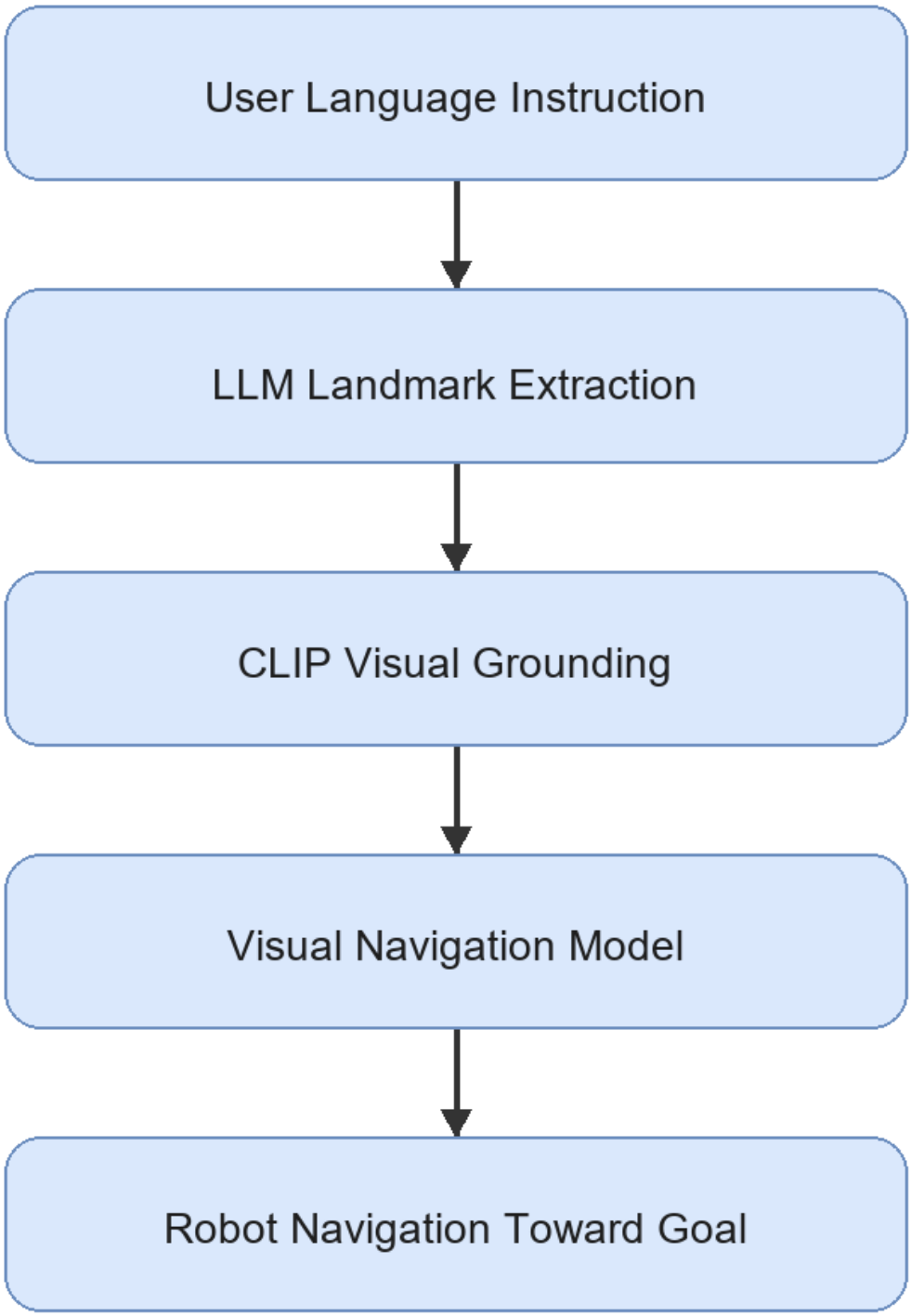
`LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action` by Shah et al. (2023a) is the main baseline for this project. LM-Nav addresses the problem that many visual navigation systems expect a goal image, while human users naturally provide language instructions. The paper avoids collecting a large language-annotated robot navigation dataset by composing pre-trained models.

The LM-Nav pipeline contains three main stages:

1. GPT-3 extracts landmarks from the natural language instruction.
2. CLIP grounds the extracted landmarks in visual observations.
3. ViNG navigates toward the selected visual goal.

This pipeline is shown in Figure 2.2.

Figure 2.2: LM-Nav baseline pipeline



2.5.2 Strength of LM-Nav

The main strength of LM-Nav is its modular architecture. It does not require one end-to-end model that directly maps language and images to motor commands. Instead, it separates

language understanding, visual grounding, and action execution. This separation is useful for research because each module can be tested and improved independently.

LM-Nav is also important because it demonstrates that pre-trained models can be combined for robot navigation without training a new language-conditioned navigation model from scratch. This idea is suitable for an FYP because collecting a large robot dataset is not realistic within the time and cost constraints of a student project.

2.5.3 Limitation of LM-Nav

LM-Nav also has limitations that motivate this project. The language module mainly extracts landmarks. This may be insufficient for instructions that include spatial relations, route order, or constraints. For example, "move toward the chair near the table" should not be reduced only to `chair` and `table`; the relation `chair near table` is also important.

Another limitation is hardware and navigation complexity. LM-Nav was demonstrated with a more capable outdoor robot platform and a visual navigation model. A student FYP setup using a Raspberry Pi 4, ESP32 sensor and motor control, USB webcam, two ultrasonic sensors, and four DC motors cannot reproduce the full navigation capability directly. Therefore, this project uses LM-Nav as an architectural inspiration, not as a full implementation target.

2.5.4 Relevance to This Project

LM-Nav provides the central structure for the proposed prototype: language understanding, visual grounding, and action execution. The difference is that this project adapts the final execution stage into a simple action-control system. Instead of using a full visual navigation policy, the Raspberry Pi 4 chooses one of a small set of actions, checks obstacle status from the ESP32 ultrasonic module, and sends validated commands to the ESP32 motor-control layer. This makes the project realistic for a low-cost indoor robot while preserving the research value of structured prompt engineering.

2.6 VLMnav as a Prompt-Based Action Selection Reference

`VLMnav: End-to-End Navigation with Vision Language Models` by Goetting et al. (2024) is highly relevant because it formulates navigation as a question-answering problem. Instead of requiring the model to output a long navigation plan, the system can ask a VLM to choose among possible actions. This is close to the needs of the

proposed robot because the first prototype only needs a small action set: `move_forward`, `turn_left`, `turn_right`, `stop`, and `search`.

The strength of VLMnav is that it connects visual observation, language instruction, and action selection through prompting. This supports the idea that prompt design can affect not only landmark extraction but also robot movement decisions. For example, a prompt can ask whether the current webcam image contains the target, whether the robot should turn to search, or whether it should stop because the target has been found.

The limitation is that direct VLM action selection may not be safe enough for unsupervised robot control. VLM outputs can be inconsistent, and the model may choose an action even when the target is not visible or the instruction is ambiguous. For this reason, the proposed FYP will validate the structured output, uncertainty level, model latency, and ESP32 ultrasonic obstacle status before physical motor execution. VLMnav is therefore treated as a prompt/action-selection reference, while the robot remains a controlled research prototype.

2.7 Spatial Grounding Methods: VLMaps and HOV-SG

Spatial grounding methods improve the connection between language and the physical environment. Two important examples are `VLMaps` and `HOV-SG`.

`VLMaps: Visual Language Maps for Robot Navigation` by Huang et al. (2023) builds an open-vocabulary visual-language map. Instead of matching a text query only to the current camera image, the robot can query a map that stores semantic visual features. The strength of VLMaps is that it provides spatial memory and can connect language to locations. Its limitation is that it requires map construction, camera pose information, and RGB-D or mapping support. This makes it more complex than the first version of the proposed FYP robot.

`HOV-SG: Hierarchical Open-Vocabulary 3D Scene Graphs for Language-Grounded Robot Navigation` by Werby et al. (2024) uses a hierarchical 3D scene graph to represent floors, rooms, and objects. This is powerful for long-horizon and multi-room navigation because the robot can reason about object and room relationships. However, it requires segmentation, 3D perception, graph construction, and more advanced sensing. This is not suitable for the first low-cost prototype, but it is valuable as a future improvement direction.

Both papers show that spatial grounding is important. They also highlight a gap for this project: a low-cost robot without RGB-D mapping still needs a way to preserve spatial relations from language. Structured prompt engineering can support this by extracting fields such as `target`, `landmarks`, and `spatial_relation`, even if the first implementation only uses webcam-based grounding.

2.8 Navigation Execution Models: ViNT and NoMaD

The execution stage determines how a robot physically moves toward a target. In advanced systems, this may involve a learned navigation policy, obstacle avoidance, and goal-conditioned control. `ViNT` and `NoMaD` are important because they represent stronger navigation execution methods than a simple rule-based motor command system.

`ViNT: A Foundation Model for Visual Navigation` by Shah et al. (2023b) proposes a visual navigation foundation model trained across diverse robot datasets. Its strength is generalization across platforms and environments. Its limitation for this FYP is that deploying and validating a full visual navigation model is outside the main scope. This project focuses on prompt engineering, visual grounding, and simple action control, not training or deploying a foundation navigation policy.

`NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration` by Sridhar et al. (2023) uses a diffusion-based policy for goal-directed navigation and exploration. Its strength is stronger navigation behavior and exploration ability. Its limitation is that it requires a more complex policy deployment pipeline and suitable robot data. For the proposed project, NoMaD is therefore best treated as future work after the basic Raspberry Pi 4 physical robot pipeline is functioning.

These works are important because they show how the simple action-control layer in this FYP could later be replaced by a stronger navigation model. However, they are not the main implementation because the research focus is prompt-engineered interpretation of natural language instructions.

2.9 Recent VLM and VLA Navigation Systems

Recent work has moved toward more integrated VLM and VLA navigation systems. These systems can reason over video, language, and action more directly than the earlier modular LM-Nav design.

`NaVid` by Zhang et al. (2024) uses video-based VLM planning for vision-and-language navigation. Its strength is that it uses temporal visual context rather than a single image. This is useful for robot navigation because the current view alone may not contain enough information. Its limitation is that video-based reasoning and model inference can be expensive for a small embedded robot.

`Uni-NaVid` by Zhang et al. (2026) extends the video-based approach toward a unified model for multiple embodied navigation tasks. Its strength is broad task coverage. Its limitation is that the system is more advanced and resource-intensive than a first FYP prototype.

`NaVILA` by Cheng et al. (2025) studies VLA navigation for legged robots. Its strength is the connection between vision-language reasoning and real robot movement. Its limitation for this project is that it focuses on more complex legged platforms and server-style model pipelines, while the proposed robot is a wheeled low-cost indoor platform.

These papers are relevant because they show the direction of future navigation research. However, they also reinforce the need for a realistic baseline prototype. A student project can first demonstrate prompt-engineered instruction parsing, webcam grounding, and simple action control before attempting advanced VLA navigation.

2.10 Critical Comparison of Related Works

The reviewed papers differ significantly in method, hardware complexity, cost, and suitability for student-level implementation. Table 2.2 compares the papers based on their main method, strength, limitation, and relevance to this project.

Table 2.2: Critical comparison of related papers

The comparison shows that LM-Nav and VLMnav are the most directly relevant to this project. LM-Nav provides the modular structure, while VLMnav supports the idea of selecting actions through prompt-based reasoning. VLMaps, HOV-SG, ViNT, NoMaD, NaVid, Uni-NaVid, and NaVILA are important, but they require additional sensing, mapping, model deployment, or compute resources that are not realistic for the first prototype.

2.11 Hardware Suitability for This FYP

A critical issue for this project is hardware feasibility. Many research systems use expensive robots, RGB-D cameras, LiDAR, server GPUs, or advanced navigation policies. This FYP

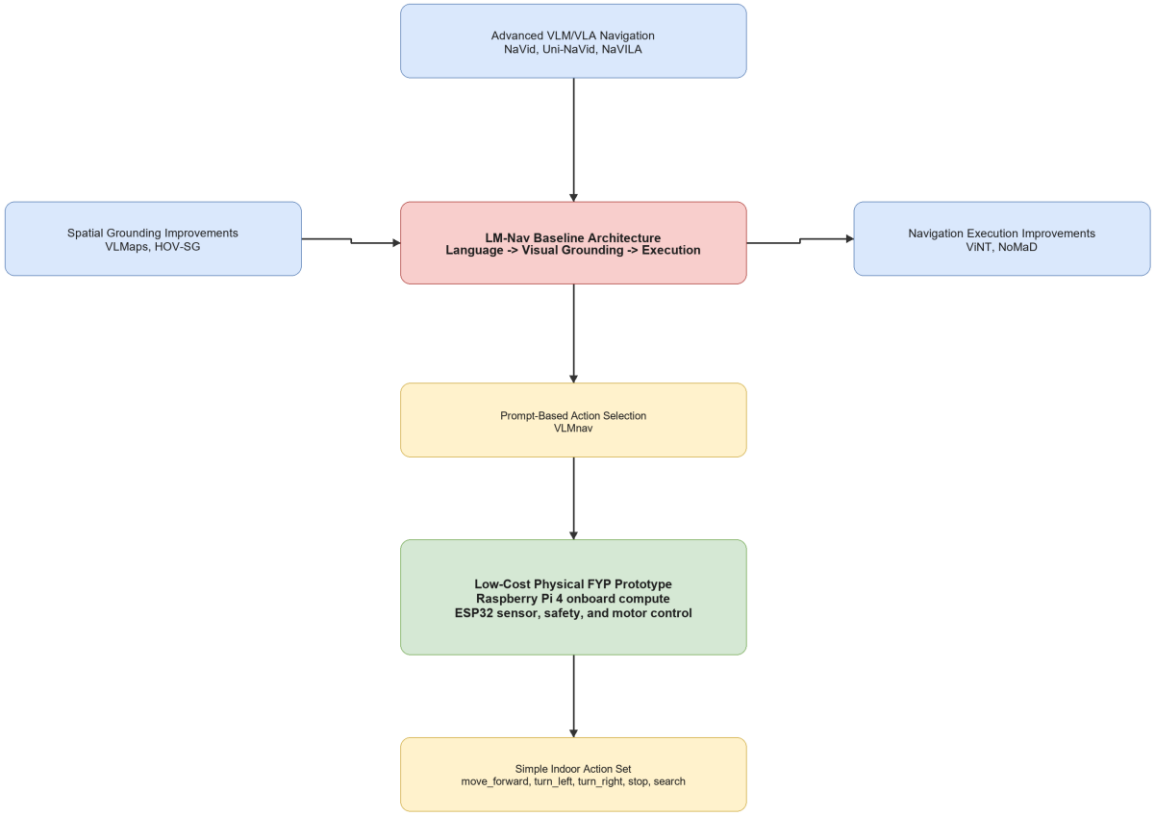
must use a lower-cost platform while still preserving a meaningful research contribution. Table 2.3 summarizes the suitability of literature components for the proposed implementation.

Table 2.3: Hardware and method suitability for this FYP

This table supports the design decision to build a low-cost indoor prototype. The project will not claim full autonomy in complex environments. Instead, it will demonstrate a baseline pipeline that future researchers can improve with better sensors, mapping, and navigation policies.

The positioning of the reviewed papers around the proposed baseline is shown in Figure 2.3.

Figure 2.3: Positioning of related works around LM-Nav



2.12 Research Gap and Motivation

The literature shows that language-guided robot navigation is an active research area, but several gaps remain for a low-cost FYP prototype.

First, existing systems can perform language-guided navigation, but many require expensive hardware, complex mapping, large models, or server-level compute. These systems are valuable research contributions, but they are not directly suitable for a student prototype using a Raspberry Pi 4, ESP32, USB webcam, two ultrasonic sensors, two MX1508 motor drivers, GY-291 accelerometer, and four DC gear motors.

Second, LM-Nav provides a strong modular baseline, but its language stage mainly extracts landmarks. Natural language instructions often include spatial relations and route information, such as "near the table", "turn left after the door", or "find the signboard". A low-cost prototype still needs to preserve this information, even if the first grounding module is simple.

Third, many advanced methods improve spatial grounding or navigation execution, but they do not directly answer how structured prompt engineering can support a simple indoor robot. VLMaps and HOV-SG show the value of maps and scene graphs, while ViNT and NoMaD show stronger navigation execution. However, this project begins earlier in the pipeline by asking how natural language can be converted into structured, validated robot action information.

Fourth, there is a need for a focused baseline physical prototype that future researchers can extend. A Raspberry Pi 4 robot with webcam grounding, ESP32 safety control, acceleration and tilt feedback, and simple actions provides a practical platform for later improvements such as RGB-D sensing, LiDAR, scene graph mapping, ViNT, NoMaD, or advanced VLM/VLA models.

Table 2.4: Research gap synthesis and project strategy

Based on these gaps, the project is motivated by the following research direction:

How can structured prompt engineering support visual grounding and simple action selection for a low-cost indoor mobile robot navigation prototype?

2.13 Chapter Summary

This chapter reviewed literature related to prompt engineering for mobile robot navigation. LM-Nav was identified as the main baseline because it provides a clear modular architecture of language understanding, visual grounding, and navigation execution. VLMnav was

reviewed as the main prompt/action-selection reference because it frames navigation as a question-answering or action-choice task.

The chapter also reviewed VLMaps and HOV-SG as spatial grounding improvements, ViNT and NoMaD as stronger navigation execution models, and NaVid, Uni-NaVid, and NaVILA as advanced VLM/VLA navigation directions. These works show the potential of language-guided navigation, but they also reveal limitations related to cost, sensing requirements, compute resources, and implementation complexity.

The research gap is that there is a need for a low-cost indoor physical prototype that studies structured prompt engineering for simple mobile robot navigation. Chapter 3 explains how this project addresses the gap using one Raspberry Pi 4 implementation with an ESP32 motor, sensor, and safety layer.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Introduction

This chapter presents the finalized methodology for `Prompt Engineering for Mobile Robot Navigation`. The project contains one physical implementation: a Raspberry Pi 4 mobile robot with an ESP32 sensor, safety, and motor-control layer.

The methodology is inspired by the modular structure of LM-Nav and the prompt-based action-selection idea of VLMnav. Language interpretation, visual grounding, and navigation execution are separated so each stage can be tested. The first prototype targets `find the green marker` and restricts all output to:

- `move_forward`
- `turn_left`
- `turn_right`
- `search`
- `stop`

The laptop-only Docker demonstration is used only to produce expected-results evidence before physical testing. It is not a second physical approach.

3.2 Research Design and Project Workflow

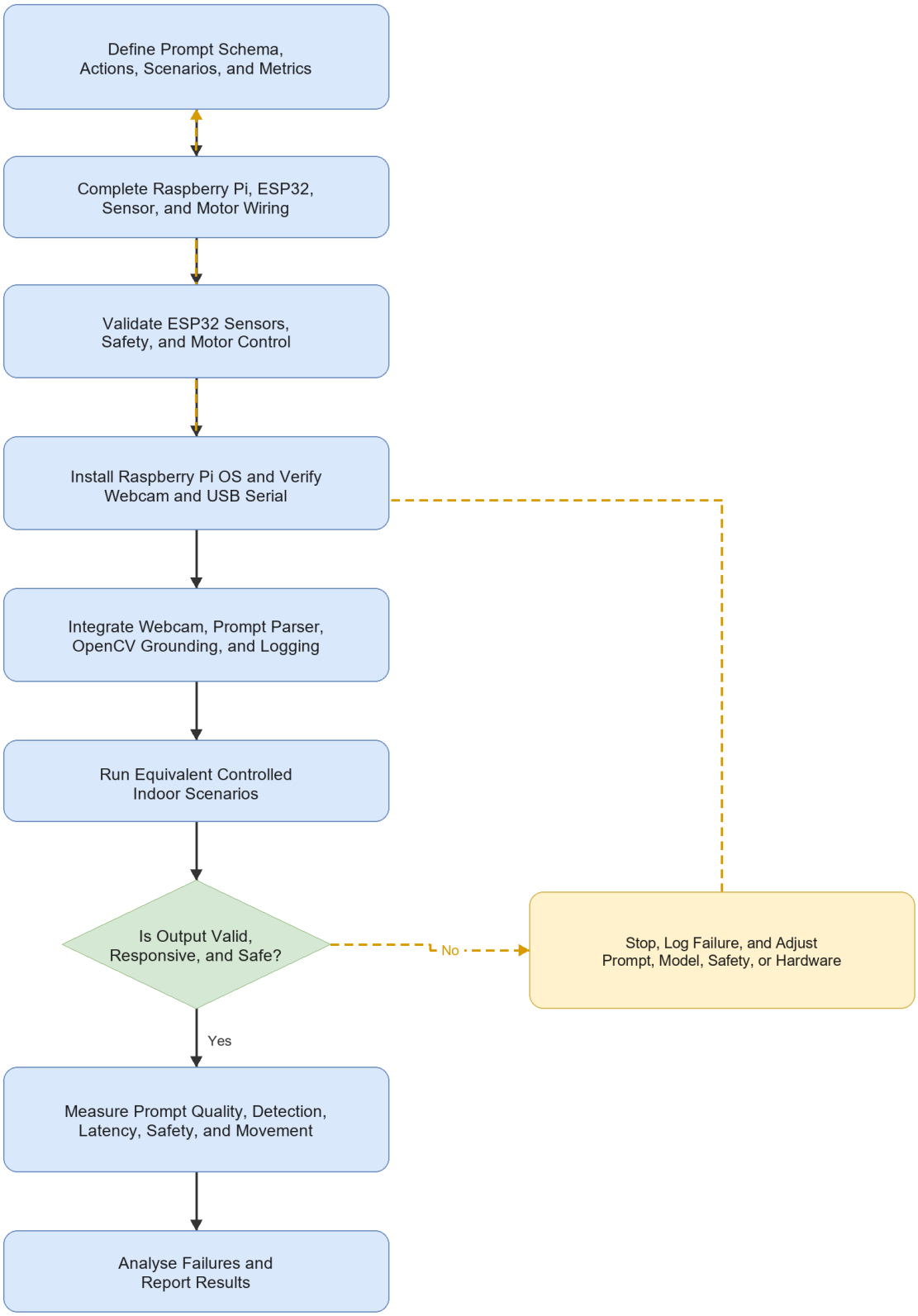
The project follows an iterative prototype methodology. The already-built mechanical and power systems are completed by wiring and validating the ESP32 sensor and motor layer, connecting Raspberry Pi USB serial, verifying webcam capture and OpenCV visual grounding, and testing one complete basic goal.

3.2.1 Methodology Alignment with Research Objectives

Table 3.1: Methodology Alignment with Research Objectives

3.2.2 Overall Project Workflow

Figure 3.1: Proposed project workflow



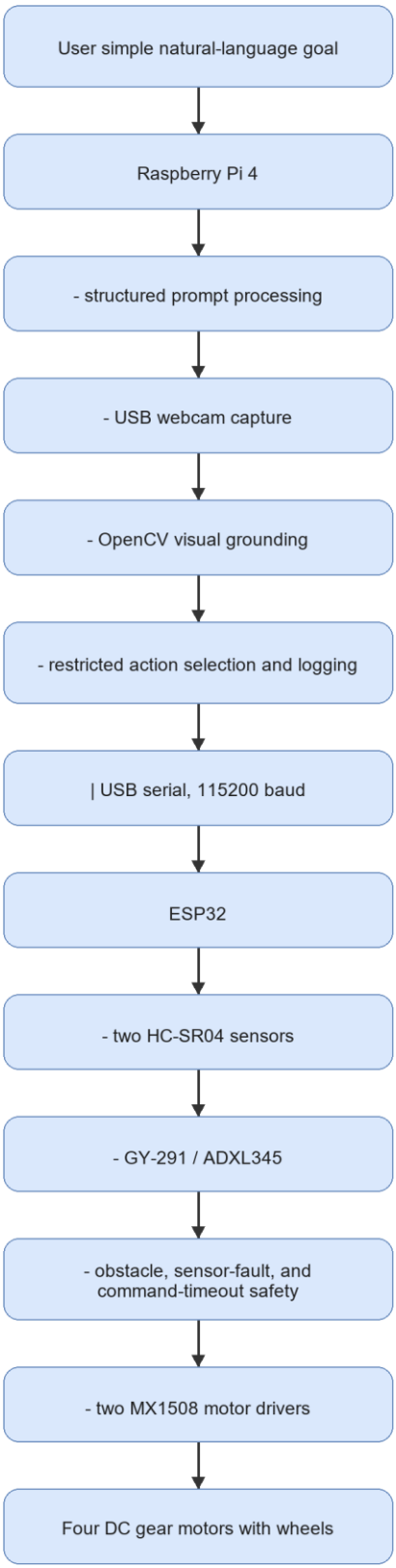
3.2.3 Development Workflow

Table 3.2: Development Workflow

3.3 Finalized System Architecture

The Raspberry Pi 4 performs high-level processing. The ESP32 handles local real-time sensing, safety, and motor control. USB serial is used between the boards because it is simple to debug and leaves all ESP32 motor GPIO pins available.

Figure 3.2: Proposed system architecture


User simple natural-language goal
Raspberry Pi 4
- structured prompt processing
- USB webcam capture
- OpenCV visual grounding
- restricted action selection and logging
| USB serial, 115200 baud
ESP32
- two HC-SR04 sensors
- GY-291 / ADXL345
- obstacle, sensor-fault, and command-timeout safety
- two MX1508 motor drivers
Four DC gear motors with wheels

3.3.1 Hardware Roles

Table 3.3: Finalized Hardware Roles

3.4 Requirements, Constraints, and Acceptance Criteria

Table 3.4: Requirements and Acceptance Criteria

The baseline does not include SLAM, LiDAR mapping, dense depth, or an unrestricted large vision-language model.

3.5 Data, Test Environment, and Experimental Materials

3.5.1 Basic Goal and Scenarios

Table 3.5: First-Prototype Scenarios

3.5.2 Experimental Materials

Table 3.6: Experimental Materials

Figure 3.3: Main hardware components for the finalized Raspberry Pi 4 robot

3.5.3 Software and Model Requirements

Table 3.7: Software, Model, and Platform Requirements

3.6 Prompt Engineering Design

The first prototype uses a deterministic structured parser. This applies the main prompt-engineering principles required by the study: fixed fields, restricted actions, explicit uncertainty, and safe rejection of unsupported goals.

Table 3.8: Structured Prompt Fields

```
{
  "instruction": "find the green marker",
  "target": "green marker",
  "landmarks": ["green marker"],
  "spatial_relation": null,
  "action_goal": "find and approach the green marker",
  "uncertainty": "low"
}
```

Unsupported or unclear instructions produce high uncertainty and `stop`.

3.7 Webcam-Based Visual Grounding and Action Selection

The Raspberry Pi captures a USB-webcam frame and detects the selected marker using OpenCV HSV colour segmentation. The largest valid marker region is used for simple action selection.

Table 3.9: Visual Grounding and Action Decision Rules

After this baseline works, the OpenCV grounding module can be replaced with a lightweight detector for chairs, doors, signboards, or other indoor landmarks.

3.8 Robot Action and Command Mapping

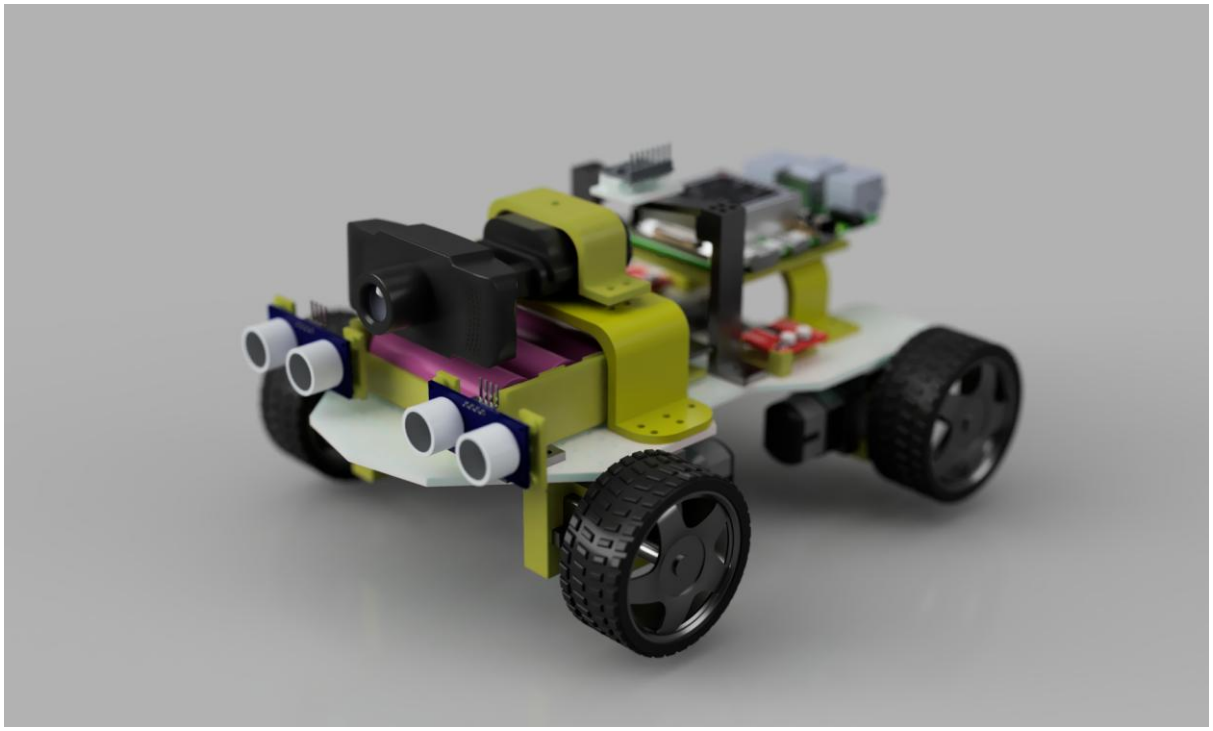
Table 3.10: Action Set and Motor Command Mapping

The ESP32 accepts only these commands. Unknown commands, loss of both ultrasonic readings, and a command delay above 1,000 ms cause `stop`.

3.9 Mechanical Design and Physical Integration

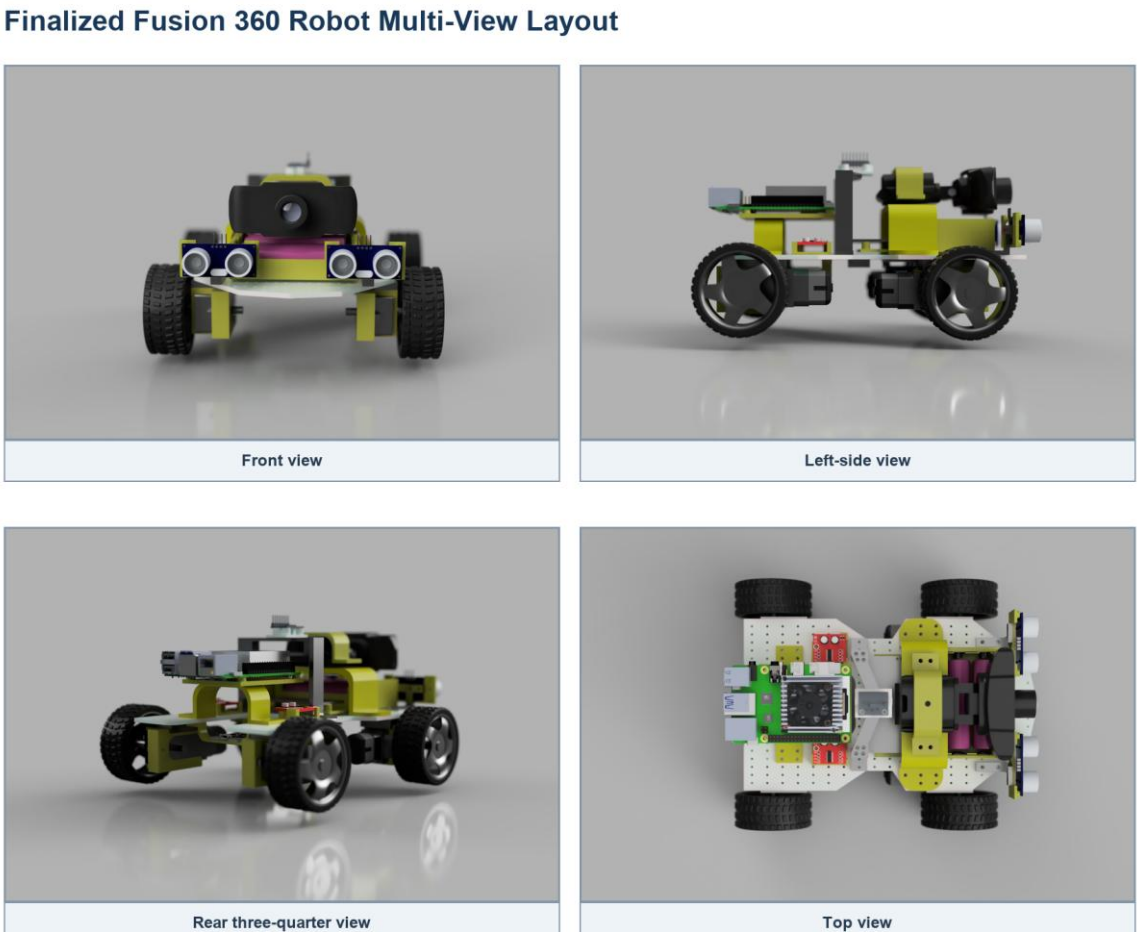
The finalized Fusion 360 robot design provides mounting positions for the Raspberry Pi 4, ESP32, USB webcam, two front ultrasonic sensors, GY-291, two MX1508 drivers, completed power system, and four motors.

Figure 3.4: Finalized Fusion 360 physical robot model



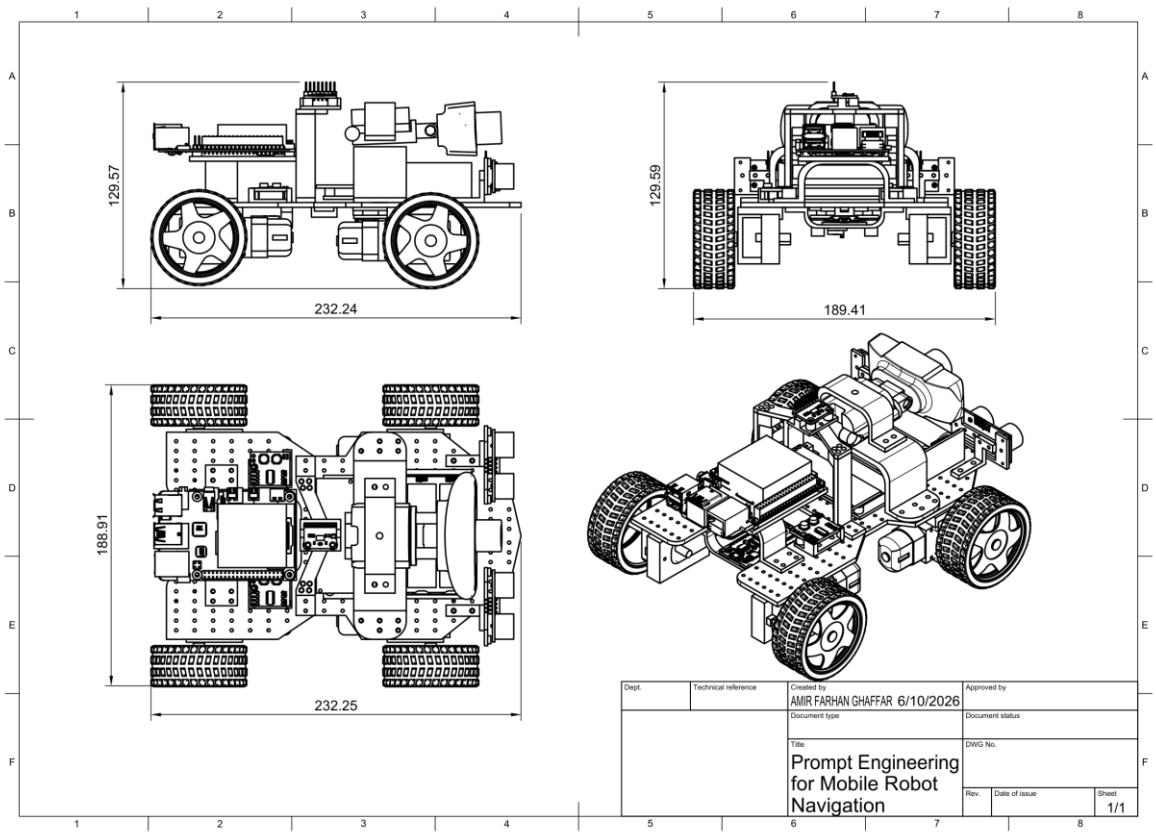
The existing render records the physical design and Raspberry Pi mounting layout.

Figure 3.5: Finalized Fusion 360 robot multi-view layout



The views support sensor visibility, wheel clearance, component spacing, cable access, and Raspberry Pi mounting verification.

Figure 3.6: Finalized mechanical design sketch and dimensions



The approximate overall dimensions are 232.24 mm long, 188.91 mm wide, and 129.57 mm high.

3.10 Finalized Wiring Architecture

Figure 3.7: Finalized component-based circuit architecture

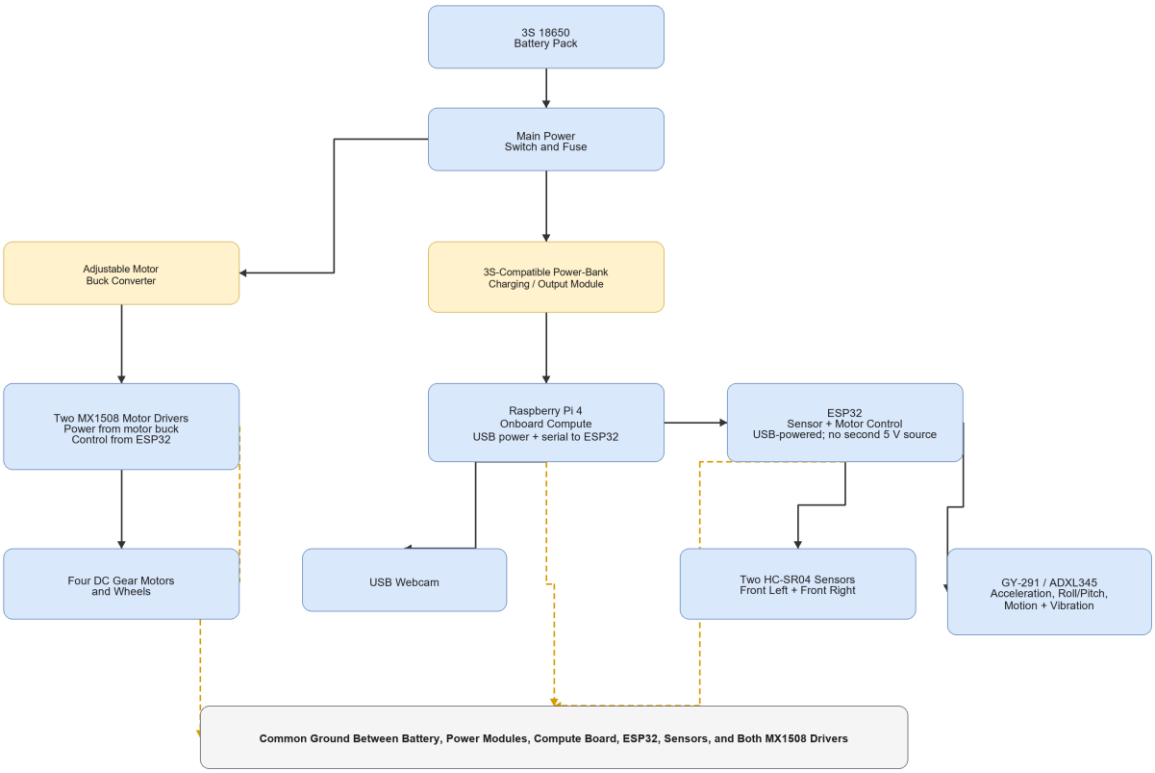
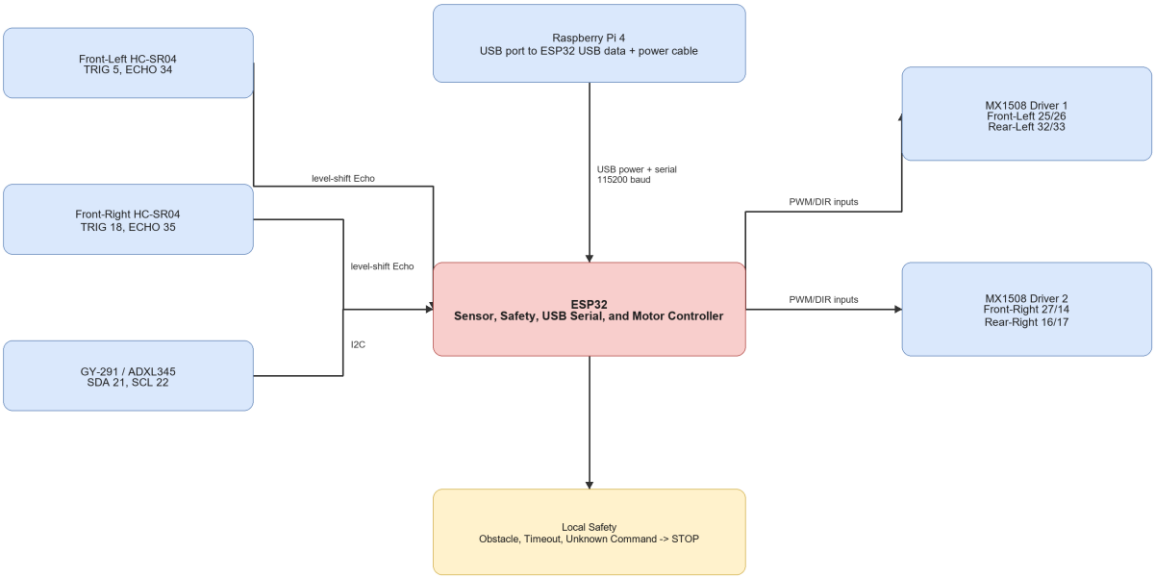


Figure 3.8: Finalized ESP32 sensor and motor wiring



The detailed connection and test sequence is provided in `docs/circuit_wiring_guide.md`.

3.11 Software Implementation

Table 3.11: Source Code and Documentation Modules

3.12 Main Equations and Decision Rules

Table 3.12: Main Equations and Decision Rules

3.13 Implementation Procedure

Table 3.13: Implementation Procedure

3.14 Testing and Validation Plan

Table 3.14: Testing and Validation Matrix

3.15 Evaluation Metrics

Table 3.15: Evaluation Metrics

3.16 Laptop-Only Docker Expected-Results Demonstration

The separate laptop demo uses a browser webcam and coloured marker to display expected navigation actions. A Docker container provides the isolated Flask, OpenCV, NumPy, and logging environment. The user manually moves the laptop; no motor interface or physical robot claim is made. Final results must come from the Raspberry Pi 4 physical robot.

3.17 Safety, Limitations, and Future Work

Testing must use the completed protected power system, fuse, reachable main switch, correct regulated rails, and a common ground. Motor tests begin with the robot raised. HC-SR04 Echo pins require level shifting.

During the first prototype, the ESP32 is powered only by the Raspberry Pi USB cable. A separate ESP32 5 V supply must not be connected simultaneously unless the USB 5 V conductor is safely isolated.

The robot stops for invalid output, high uncertainty, obstacle detection, sensor fault, unknown commands, or command timeout. Testing is supervised in a controlled indoor area.

The USB webcam does not provide dense depth or a 3D map. Two ultrasonic sensors provide limited obstacle coverage. The GY-291 does not provide yaw heading. The Raspberry Pi 4

has limited performance for large models, and the first prototype supports only simple coloured-marker goals.

Future work may include lightweight object detection, API-based VLM experiments, wheel encoders, magnetometer or full IMU, RGB-D camera, LiDAR, mapping, relation-aware prompts, and stronger navigation policies.

3.18 Summary

This chapter finalized the methodology around one Raspberry Pi 4 physical robot. The project connects structured prompt processing and OpenCV visual grounding to an ESP32 that independently handles sensors, safety, and four-motor control. The first measurable target is a reliable and safe single-goal indoor marker-navigation demonstration.

CHAPTER 4

EXPECTED RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the expected results for FYP1. The expectations are based on the finalized Raspberry Pi 4 architecture, structured prompt design, completed physical hardware, ESP32 safety logic, Docker laptop expected-results demonstration, and planned physical validation. Final measured robot results will be presented during FYP2.

4.2 Expected Basic Prototype Result

The first Raspberry Pi 4 prototype is expected to complete the simple and repeatable goal `find the green marker`. The instruction should be converted into a structured target and approved action set. OpenCV colour grounding should locate the marker in the webcam view and select an action from its image position.

Table 4.1: Summary of Expected Basic Prototype Results

4.3 Expected Docker Laptop Demonstration

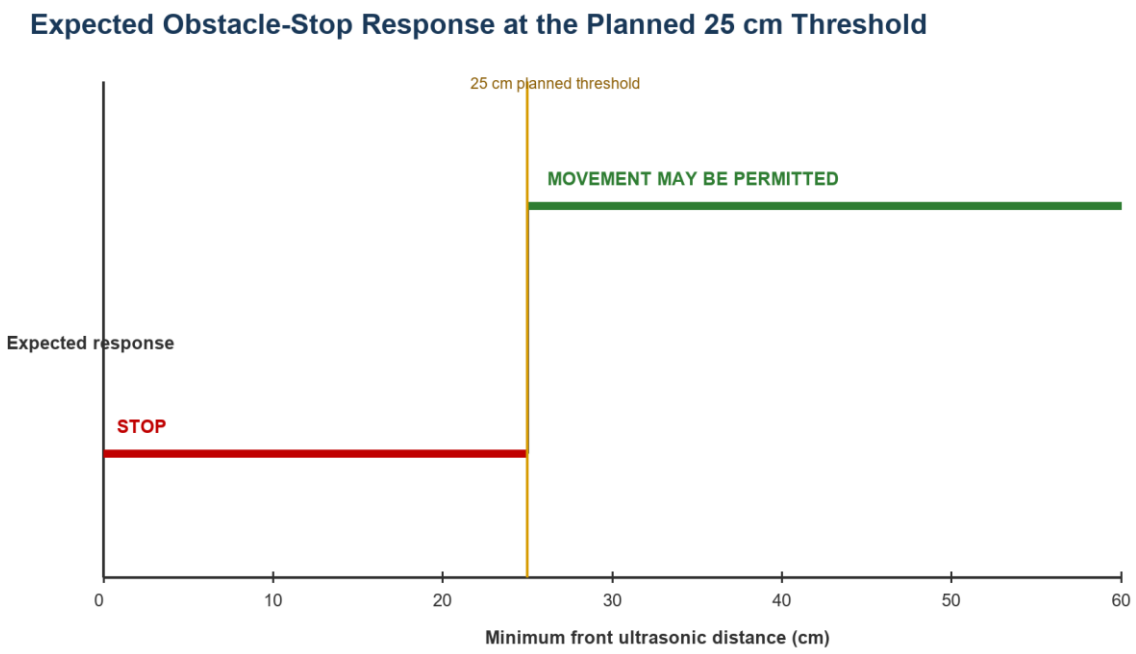
The laptop-only Docker demonstration is expected to show the same planned action interface before physical robot testing. A browser captures a coloured marker through the laptop webcam and sends frames to the containerized Python/OpenCV service. The application displays `TURN LEFT`, `TURN RIGHT`, `MOVE FORWARD`, `SEARCH`, or `STOP`, and the user manually moves the laptop according to the displayed action.

This demonstration provides expected-results evidence only. It is not a second physical implementation and is not used as the final robot result.

4.4 Expected Safety Simulations

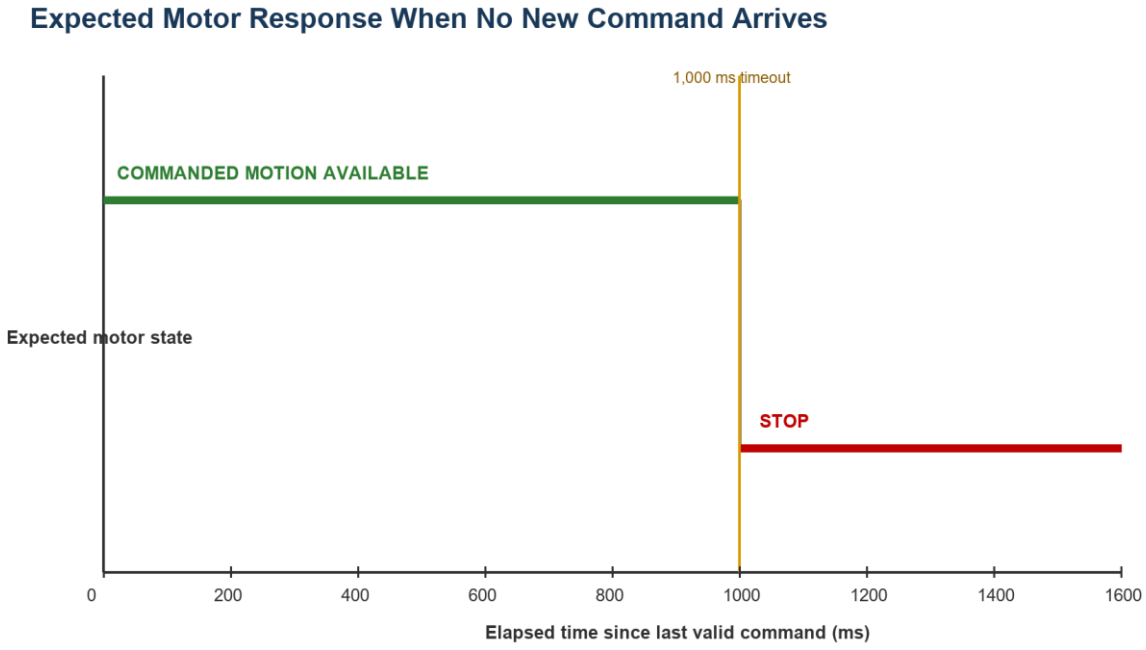
The ESP32 uses the minimum reading from the two front HC-SR04 sensors. With the planned 25 cm threshold, the robot is expected to stop when either valid sensor reading is below 25 cm.

Figure 4.1: Expected obstacle-stop response at the planned 25 cm threshold



The ESP32 also applies a 1,000 ms command timeout. If no new valid command is received within this period, all motor outputs are expected to stop.

Figure 4.2: Expected motor response when the command timeout exceeds 1,000 ms



These simulations demonstrate the intended decision logic only. The final distance threshold and physical stopping response require calibration and repeated testing during FYP2.

4.5 Required FYP2 Results

Table 4.2: Required FYP2 Measurements

4.6 Discussion

The expected results suggest that structured prompt engineering can provide a simple interface between a natural-language goal and a restricted robot action set. The Raspberry Pi 4 is expected to provide a flexible environment for Python, OpenCV, USB devices, logging, and later model experiments, while the ESP32 independently prevents movement during obstacle, sensor-fault, timeout, and invalid-command conditions.

The basic marker goal is intentionally limited. Demonstrating it reliably is more valuable than claiming complex navigation without sufficient sensor coverage, mapping, or model capability. Once the baseline is verified, the project can extend to object detection for indoor landmarks and more complex instructions.

4.7 Summary

This chapter presented the expected FYP1 outcomes for the Raspberry Pi 4 prototype, Docker laptop expected-results demonstration, and safety logic. Final physical results will measure whether the robot can safely find and approach a supported visual landmark.

REFERENCE

Cheng, A.-C., Ji, Y., Yang, Z., Gongye, Z., Zou, X., Kautz, J., Biyik, E., Yin, H., Liu, S., & Wang, X. (2025). NaVILA: Legged robot vision-language-action model for navigation. *Proceedings of Robotics: Science and Systems*. https://doi.org/10.15607/RSS.2025.XXI.018

Goetting, D., Singh, H. G., & Loquercio, A. (2024). *End-to-end navigation with vision language models: Transforming spatial reasoning into question-answering*. arXiv. https://doi.org/10.48550/arXiv.2411.05755

Huang, C., Mees, O., Zeng, A., & Burgard, W. (2023). Visual language maps for robot navigation. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. https://vlmaps.github.io/

Shah, D., Osinski, B., Ichter, B., & Levine, S. (2023a). LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action. *Proceedings of the 6th Conference on Robot Learning, Proceedings of Machine Learning Research, 205*, 492-504. https://proceedings.mlr.press/v205/shah23b.html

Shah, D., Sridhar, A., Dashora, N., Stachowicz, K., Black, K., Hirose, N., & Levine, S. (2023b). ViNT: A foundation model for visual navigation. *Proceedings of the 7th Conference on Robot Learning, Proceedings of Machine Learning Research, 229*, 711-733. https://proceedings.mlr.press/v229/shah23a.html

Sridhar, A., Shah, D., Glossop, C., & Levine, S. (2023). *NoMaD: Goal masked diffusion policies for navigation and exploration*. arXiv. https://arxiv.org/abs/2310.07896

Werby, A., Huang, C., Buchner, M., Valada, A., & Burgard, W. (2024). Hierarchical open-vocabulary 3D scene graphs for language-grounded robot navigation. *Proceedings of Robotics: Science and Systems*. https://doi.org/10.15607/RSS.2024.XX.077

Zhang, J., Wang, K., Xu, R., Zhou, G., Hong, Y., Fang, X., Wu, Q., Zhang, Z., & Wang, H. (2024). NaVid: Video-based VLM plans the next step for vision-and-language navigation. *Proceedings of Robotics: Science and Systems*. https://doi.org/10.15607/RSS.2024.XX.079

Zhang, J., Wang, K., Wang, S., Li, M., Liu, H., Wei, S., Wang, Z., Zhang, Z., & Wang, H. (2026). Uni-NaVid: A video-based vision-language-action model for unifying embodied

navigation tasks. *Robotics: Science and Systems 2026 Program*. https://roboticsconference.org/program/papers/13/